CoTFormer Reproducibility Report

Alberto Berni (ab3u21)[1], **Bryan Vullo** (bv1g22)[1], **Tevfik Aras Kavuncu** (tak1e25)[1]
[1]*University of Southampton, School of Electronics and Computer Science*

1 Introduction

Transformers replaced recurrence with parallel self-attention, enabling scalable language modelling but weakening the architectural bias toward iterative computation [1]. This tension is central to neural algorithmic reasoning: earlier recurrent models (i.e. Neural GPUs) could generalise from short training instances to longer algorithmic inputs through repeated parameter sharing [2], and Block Universal Transformers (BUT) later reintroduced this idea by repeatedly applying the same Transformer block [3].

The CoTFormer architecture formalises this connection by treating Chain-of-thought as a form of *recurrent latent computation*. CoTFormer preserves previous intermediate states as attendable representations, more closely mimicking how explicit thought tokens can attend to earlier thought tokens [4]. Authors evaluate this architecture through perplexity and compute efficiency, despite recent related work suggesting that "*perplexity, although very useful for training, is a narrow measure of model quality*", showing looped models can improve reasoning-heavy tasks despite worse perplexity and memorisation than iso-FLOP non-looped baselines [5]. This motivates evaluating CoTFormer not only as a language model, but as a recurrent architecture with a potential inductive bias for iterative reasoning. We reproduce and evaluate the main claims of CoTFormer and its variants, and extend its evaluation to ask *whether its recurrent structure improves controlled algorithmic generalisation*.

2 Methodology

Reproduction of this work[1] was performed on L4 GPUs, requiring specific adaptations to the original codebase, designed for A100 GPUs. To reconcile this divergence, we applied the following:

Minor Adaptations Reduced vRAM budget required lowering batch size and proportionally increasing gradient accumulation steps, retaining the original batch size (128). Evaluation of MAC metrics was estimated in closed-form due to these constraints.

LN-CoTFormer We initially trained this variant to 40k steps for Table 2, resuming to 60k steps for further evaluation. This invalidated the previous learning-rate scheduler, requiring use of a different one and causing the 0.4 perplexity divergence observed in Table 3.

ADM Distributed Training The original repository bypassed RNG state check-pointing (required by the ADM). Replacing the NCCL backend with Gloo guaranteed RNG state restoration while confounding training dynamics (section 3). For other experiments NCCL's deadlock and RNG state checkpointing issues were fixed.

3 Reproduction

Base CotFormer Base CoTFormer models are reproducible; we notice "*benign spikes*" during training, later explored in section 4.1.

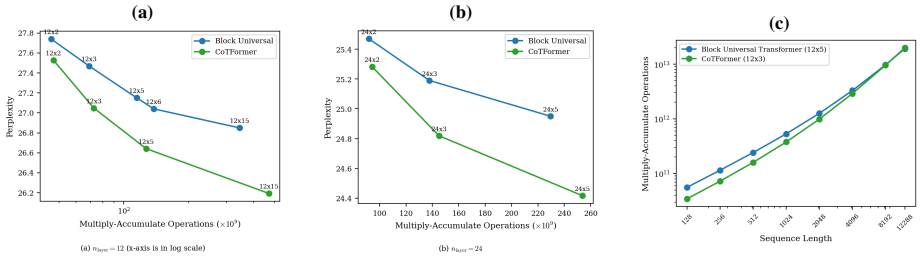

Figure 1: **(a)-(b)** Reproduced comparison of BUT (original results) and CotFormer pareto curves; **(c)** Reproduced comparison of compute intensity when performance is comparable. (Paper fig. 1a, 1b, 2)

[1]Recorded on GitHub, forked from the original.

Model	Base Layers (n_{layer})	Theirs n_{repeat}			Ours n_{repeat}			Δ (Ours − Theirs) n_{repeat}		
		2	3	5	2	3	5	2	3	5
CoTFormer	12	**27.55 (0.02)**	**27.07 (0.01)**	**26.64 (0.04)**	**27.53 (0.67)**	**27.05 (0.65)**	**26.64 (0.64)**	−0.02	−0.02	0.00
CoTFormer	24	**25.28 (0.00)**	**24.85 (0.04)**	**24.48 (0.03)**	**25.28 (0.61)**	**24.82 (0.59)**	**24.42 (0.58)**	0.00	−0.03	−0.06

Table 1: Perplexity comparison. *Theirs*: SEM over 3 seeds (paper Table 1). *Ours*: single seed; uncertainty is per-batch CI_{95}. Δ shows the difference in perplexity (Ours − Theirs); negative values means our run is better.

CoTFormer Variants The original CoTFormer architecture is extended by three compounding variants: **i.** a standard CoTFormer that reserves first and last layers for embedding translation alone **ii.** LN-CoTFormer adds a post-repeat LayerNorm to stabilise the recursive residual stream, and **iii.** Adaptive Depth Module (ADM) to dynamically route token computation and reduce recurrence during inference. We reproduce the variants below and evaluate their structural hypotheses.

Variant	Layers (effective)	Paper PPL (SEM)	Ours PPL (SEM)	Δ	95% CI (ours)
CoTFormer	24×5 (120)	24.48 (0.03)	24.42	−0.06	[23.84, 25.01]
CoTFormer + Reserved	$24, 2 \to 21 \times 5 \to 1$ (108)	24.51 (0.01)	24.64	+0.13	[24.25, 25.03]
LN-CoTFormer (40k)	$24, 2 \to 21 \times 5 \to 1$ (108)	24.11 (0.03)	24.13	+0.02	[23.75, 24.51]

Table 2: Reproduction of the CoTFormer's Table 2 results at 40k training steps.

Reserved Layers & LN-CoTFormer CoTFormer with Reserved Layers reproduces its original results within statistical bounds (Table 2). This change alone[2] *isn't* more parameter-efficient than the baseline Transformer (48L) nor the CoTFormer (24×5) on perplexity alone . The 24-layer model effectively executes a 108-layer forward pass, showing it can *approximate* deeper standard model capacity, paying a cost in sheer performance. **LayerNorm:** Authors justify this variant claiming "*it is important to maintain a consistent input's scale (across repeats)*"; the insight follows from the original architecture's [1] and remains empirically valid given the 0.51 PPL improvement from the baseline CoTFomer. Crucially, the variant outperforms the 48-layer Transformer baseline, a much more widely adopted architecture, by 0.29 PPL. While the result shows recursive computation (similarly to parallel counterparts) benefits from explicit feature scaling to prevent representation drift, it fails to explain *why* this is the case.

ADM CoTFormer By employing a "*Mixture of Repeats*" strategy with randomized capacities, the ADM trains a routing mechanism to decide whether a token requires further recurrent processing [4]. While the original Figure 4 (fig. 2**(a)**) can be replicated, the core claim that "*the ADM can efficiently route computation based on token difficulty*" cannot be confirmed as it lacks a clear definition of efficiency: showing the ADM outperforms its fixed-compute-budget copy is hardly a proof of this.

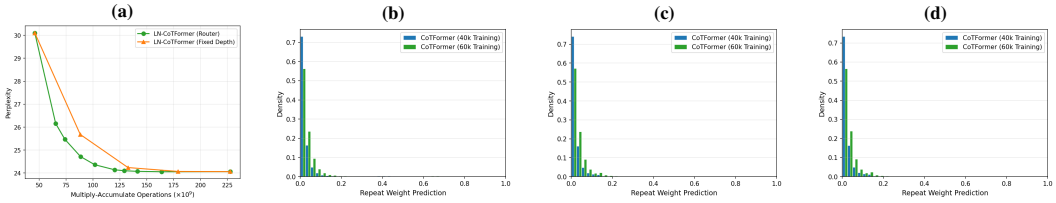

Figure 2: **(a)** Reproduced Pareto curve of compute vs perplexity; **(b)-(d)** Comparison of router weight extraction methods (raw, per-index cascade, and per-token cascade) at 40k and 60k training steps.

Irreproducible Authors claim the adaptive model "*would benefit more from longer training*", proposing extended training budgets. Using Figure [4], they argue the model "*increasingly utilises deeper repeats over time*". The claims are unrelated: increased usage of final repeats only proves the model allocates more compute; verifying a predictive benefit requires controlled evaluation at matched inference compute (i.e. fig. 1). Further, the authors' extraction script contains an inaccuracy

[2]Isolating recurrent reasoning in a central block by reserving the first and last layers strictly for embedding translation.

in how it orders extracted weights (`get_router_weights.py`, 129-131), whereby the intended "*final repeat*" becomes a cascaded minimum across repeats. We replicate this exactly in fig. 2 **(c)**, further evaluating two different extraction methods (fig. 2): **(b)** the raw router weights, and **(d)** corrected per-token cascade. Across all histograms, densities decay drastically compared to the original figure: ADM final repeat usage is lower than claimed. This divergence is confounded by back-end divergences; we can qualitatively confirm a *marginal* increase in final repeat usage in longer training, but cannot conclude whether the claimed distribution is genuine or a by-product of our constraints.

Model	**Paper PPL**	**Ours PPL (SEM)**	**Δ**
LN CoTFormer 60k	23.19	23.59 (0.01)	+0.40
ADM 60k (full compute)	23.83	24.06 (0.01)	+0.23
Adaptive cost	+0.64	+0.47	N/A

Table 3: Reproduced CoTFormer Section 5 PPL[4] comparison at 60k training steps.

4 Extensions

4.1 Phase change analysis

We notice a significant shift in attention distribution across CoTFormer repeat states during training (fig. 3). Around 27k training steps, attention mass redistributes rapidly across different repeat budgets, accompanied by "*benign spikes*" in gradient norm and a substantial increase in within-repeat attention entropy for certain repeats. We interpret this as *faint* evidence of repeat-state specialisation: middle blocks may learn non-uniform uses of latent repeat states. Establishing if these correspond to algorithmic recurrent reasoning requires additional experiments, which we leave to future work.

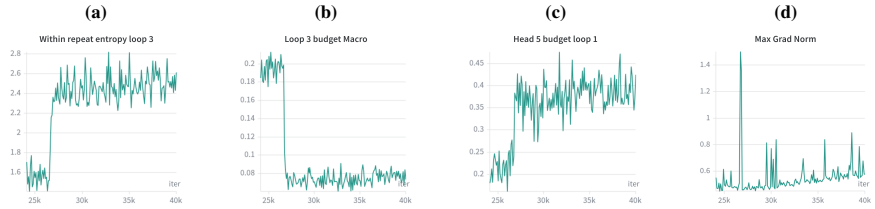

Figure 3: Repeat-state attention reorganisation during OWT2 training of a 2→24×5→1 CoTFormer.

4.2 Shifted Start and P-hop

To answer our research question (section 1) we adapt the shifted-start counting task from Chang and Bisk [6], extending the limited training split with variable-length examples below the maximum training length. Early high-width runs were unstable and prone to overfitting, so we reduced width to 128 and used 4 heads while keeping the expanded dataset. Across BUT, CoTFormer and Transformer baselines, OOD accuracy remained capped around 0.26–0.27. Increasing loop depth did not improve extrapolation under teacher-forced sequence prediction, suggesting that models may rely on *positional shortcuts* rather than an iterative counting procedure. The strongest single-seed result came from the reserved-layer CoTFormer, although the effect is seed-sensitive.

Model	Seeds	Mean OOD acc	Std	Max OOD acc
CoTFormer: $1 \to 1\times3 \to 1$	4	**0.267977**	0.009061	**0.280910**
CoTFormer: 1×8	2	0.259697	0.003039	0.261846
CoTFormer: 1×4	4	0.261681	0.003881	0.266346
Base: 4×1	4	0.267383	0.002967	0.270341
Base: 1×1	4	0.234779	0.003352	0.238078

Table 4: Shifted-start OOD accuracy.

Model	Eff. depth	Step	Val acc	Test acc
Base Transformer 1×1	1	240k	0.4659	0.4676
Base Transformer 6×1	6	300k	0.6121	0.6133
CoTFormer 1×4	4	248k	0.5935	0.6005
BUT 1×6	6	250k	0.6582	0.6597
CoTFormer 1×6	6	300k	**0.6627**	**0.6619**
CoTFormer $1 \to 1\times4 \to 1$	6	272k	0.6605	0.6585

Table 5: Single-seed p-hop induction results on constructive $p = 32$, $n = 256$, $|\Sigma| = 4$.

Unlike shifted-start inductive counting, the p-hop task benefits from repeated computation. In shifted-start counting, increasing loop depth did not improve OOD extrapolation suggesting that recurrence over layers did not induce a true counter-like procedure, while on p-hop induction, looped models clearly outperform shallow Transformer baselines on this run. The 1×1 Transformer reaches only 0.4676 test accuracy, while 1×6 BUT and CoTFormer reach roughly 0.66. This supports a task-dependent interpretation: looped computation appears useful when the target algorithm resembles repeated retrieval or pointer chasing, but it does not automatically provide the successor-state mechanism required for inductive counting.

The CoTFormer-specific advantage is modest. CoTFormer 1×6 slightly outperforms BUT, and the reserved-middle variant $1 \to 1\times4 \to 1$ performs similarly. These results support the usefulness of repeated latent computation for p-hop induction, but do not isolate a large benefit of CoTFormer's cache mechanism over vanilla block looping.

5 Conclusion

We largely reproduce the main CoTFormer perplexity results, including the benefit of post-repeat LayerNorm, but find the adaptive-depth claims less clean: ADM preserves a compute–perplexity tradeoff, while its reported deep-repeat usage is sensitive to extraction method and training confounders. Our extensions suggest a narrower interpretation of CoTFormer's recurrent bias. Repeated latent computation helps on p-hop induction, where the target resembles iterative retrieval, but does not by itself induce robust OOD counting on shifted-start. Thus, CoTFormer appears promising as a compute-sharing architecture with task-dependent algorithmic benefits, rather than a general mechanism for latent chain-of-thought reasoning. Stronger claims require matched-compute evaluations, more seeds, and controlled tasks isolating when recurrence learns an actual iterative procedure.

6 References

[1] Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[2] Kaiser and Sutskever, "Neural gpus learn algorithms," in *International Conference on Learning Representations (ICLR)*, arXiv:1511.08228, 2016.

[3] M. Dehghani, S. Gouws, O. Vinyals, J. Uszkoreit, and Ł. Kaiser, "Universal transformers," in *International Conference on Learning Representations (ICLR)*, 2019.

[4] A. Mohtashami, M. Pagliardini, and M. Jaggi, "Cotformer: A chain-of-thought driven architecture with budget-adaptive computation cost at inference," in *The Thirteenth International Conference on Representation Learning (ICLR)*, 2025. [Online]. Available: https://openreview.net/forum?id=7igPXQFupX.

[5] Saunshi, Dikkala, Li, Kumar, and Reddi, "Reasoning with latent thoughts: On the power of looped transformers," in *International Conference on Learning Representations (ICLR)*, arXiv:2502.17416, 2025.

[6] Chang and Bisk, "Language models need inductive biases to count inductively," in *International Conference on Learning Representations (ICLR)*, 2025.